CMSC131: Interfaces and Inheritance and Recursive Tracing

Chapter 12

By Naren Ayinala

May 2026

Contents

1 The Big Picture: Related Objects, Shared Behavior **3**

2 Inheritance: Start with a Parent Class **3**

2.1 The `extends` Keyword 3

2.2 Picture the Relationship 4

2.3 What Is Inherited? What Is Not? 4

3 Constructors and the `super` Keyword **4**

3.1 Constructor Call Order 5

4 Method Overriding: A Child Changes Behavior **5**

4.1 Calling the Parent Version with `super.method()` 6

4.2 Overriding Versus Overloading 6

5 Polymorphism: One Reference, Different Objects **7**

5.1 A Reference Type Is Not the Object Type 7

5.2 Upcasting, Downcasting, and `instanceof` 7

6 Why Polymorphism Is Useful: Arrays of Different Subclasses **8**

7 Interfaces: A Contract for Behavior **8**

7.1 Visualizing an Interface Contract 9

7.2 A More Practical Interface: `Payable` 9

8 Interface Rules Students Should Know **10**

9 Abstract Classes: Between Regular Classes and Interfaces 10

9.1 Class Versus Abstract Class Versus Interface 11

10 The `Object` Class: A Parent of Every Class 11

10.1 Reference Equality Versus Logical Equality 12

11 Static Methods and Fields Are Not Dynamically Overridden 12

12 A Complete Example: Shapes 13

13 Common Mistakes and How to Debug Them 14

13.1 Mistake 1: Calling a Child-Only Method Through a Parent Reference 14

13.2 Mistake 2: Forgetting the Required Parent Constructor 14

13.3 Mistake 3: Accidentally Overloading Instead of Overriding 14

13.4 Mistake 4: Making an Interface Implementation Nonpublic 14

13.5 Mistake 5: Assuming a Cast Changes the Object 15

14 Tracing Practice with Answers 15

14.1 Trace A: Which Method Runs? 15

14.2 Trace B: Constructor and Override 15

14.3 Trace C: Interface References 15

15 Final Reference: Key Rules 16

1 The Big Picture: Related Objects, Shared Behavior

Imagine writing a program for a car rental company. The program stores cars, motorcycles, and trucks. Every vehicle has a license plate and a maximum speed, but different kinds of vehicles may describe themselves differently. One approach is to copy and paste fields and methods into every class. That works for a tiny example, but repeated code is difficult to maintain: a change must be made in many places.

Java offers three connected ideas:

- **Inheritance** lets a new class reuse and specialize an existing class. A `Car` *is a* `Vehicle`.
- An **interface** describes a capability that different classes promise to provide. A car and a phone might both be `Chargeable`, even though a phone is not a vehicle.
- **Polymorphism** lets code use a common type while Java selects the appropriate implementation for the actual object.

These concepts work together, but they answer different questions. Inheritance describes an *is-a relationship*; interfaces describe a *can-do contract*; polymorphism describes *how one reference type can work with many kinds of objects.*

2 Inheritance: Start with a Parent Class

2.1 The `extends` Keyword

Consider the following two classes. The parent class holds behavior common to all vehicles, while the child adds car-specific behavior.

```
class Vehicle {
    private String plate;

    public Vehicle(String plate) {
        this.plate = plate;
    }

    public String getPlate() {
        return plate;
    }

    public void start() {
        System.out.println("Vehicle starting");
    }
}

class Car extends Vehicle {
    private int doors;

    public Car(String plate, int doors) {
        super(plate);
        this.doors = doors;
    }

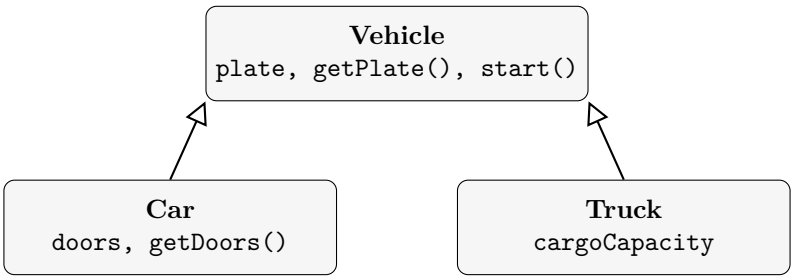
    public int getDoors() {
```

```
        return doors;
    }
}
```

The declaration `class Car extends Vehicle` says that `Car` is a **subclass** (child) of `Vehicle`, the **superclass** (parent). A `Car` object can use inherited accessible methods such as `getPlate()` and `start()`, even though those methods were not rewritten in `Car`.

```
Car car = new Car("MD-123", 4);
System.out.println(car.getPlate()); // MD-123
System.out.println(car.getDoors()); // 4
car.start(); // Vehicle starting
```

2.2 Picture the Relationship



The arrows point toward the more general type. A car is a vehicle, and a truck is a vehicle. But a vehicle is *not necessarily* a car. Inheritance is a one-way relationship.

2.3 What Is Inherited? What Is Not?

A subclass inherits accessible instance methods and fields as members of its class hierarchy. However, **private fields are not directly accessible in subclass code**. The `Car` class cannot write `plate = "ABC";` because `plate` is private to `Vehicle`. It can use a public or protected method provided by `Vehicle`. Constructors are **not inherited**; each class declares its own constructors.

The `protected` modifier allows access in subclasses (subject to Java's package and access rules). For introductory code, prefer `private` fields with carefully designed public or protected methods, rather than making fields public.

3 Constructors and the `super` Keyword

When a `Car` object is constructed, its `Vehicle` portion must be initialized first. A subclass constructor uses `super(...)` to call a superclass constructor.

```
class Animal {
    private String name;

    public Animal(String name) {
        this.name = name;
        System.out.println("Animal constructed");
    }
```

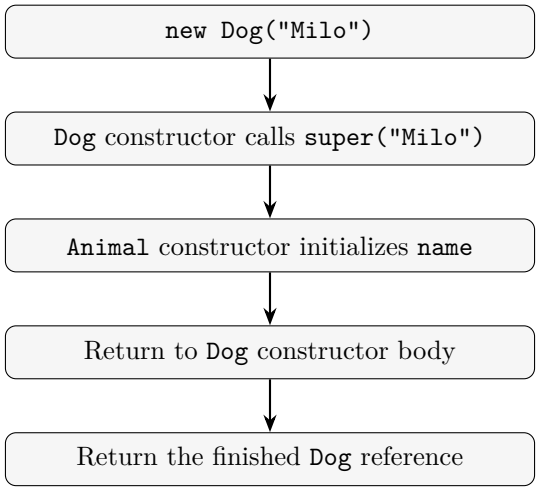
```
    public String getName() { return name; }
}

class Dog extends Animal {
    public Dog(String name) {
        super(name); // Calls Animal(String)
        System.out.println("Dog constructed");
    }
}

Dog d = new Dog("Milo");
// Animal constructed
// Dog constructed
```

Important rules: An explicit `super(...)` must be the first statement in a constructor (in the introductory Java syntax taught here). If omitted, Java tries to insert `super()` automatically. That works only if the superclass has an accessible no-argument constructor. If the parent only has `Animal(String)`, the child must explicitly call an appropriate parent constructor. `this(...)` calls another constructor in the *same* class; `super(...)` calls one in the *parent* class. You cannot use both as the first constructor statement.

3.1 Constructor Call Order



4 Method Overriding: A Child Changes Behavior

Inheritance is useful even when the child needs a different version of a parent method. **Overriding** means declaring an instance method in a subclass with the same method signature and a compatible return type, replacing the inherited implementation for objects of that subclass.

```
class Animal {
    public void speak() {
        System.out.println("Some animal sound");
    }
}

class Dog extends Animal {
```

```
    @Override
    public void speak() {
        System.out.println("Woof!");
    }
}

class Cat extends Animal {
    @Override
    public void speak() {
        System.out.println("Meow!");
    }
}
```

`@Override` is an annotation asking the compiler to check that the method really overrides an inherited method. It helps catch misspellings and incorrect parameter lists. The annotation does not itself perform overriding.

```
Dog dog = new Dog();
dog.speak(); // Woof!
```

4.1 Calling the Parent Version with `super.method()`

A subclass can extend the parent's behavior rather than completely replace it:

```
class Vehicle {
    public void inspect() {
        System.out.println("Check tires");
    }
}
class Car extends Vehicle {
    @Override
    public void inspect() {
        super.inspect(); // Run the parent's method
        System.out.println("Check seat belts");
    }
}
// new Car().inspect() prints both lines.
```

`super.inspect()` explicitly invokes the superclass implementation. Contrast this with `this.inspect()`, which would invoke the current object's overridden method again and might cause infinite recursion.

4.2 Overriding Versus Overloading

	Overloading	**Overriding**
Where?	Usually same class; may involve inherited methods	Subclass and superclass
Parameters	Must differ in number or types	Must match the inherited method signature
Decision	Compile time, using argument types	Runtime, using actual object type
Example	`print(int)` and `print(String)`	`Dog.speak()` replaces `Animal.speak()`

Common mistake: Writing `speak(String sound)` in `Dog` does not override `speak()`. It creates a different, overloaded method. Use `@Override` to detect this.

5 Polymorphism: One Reference, Different Objects

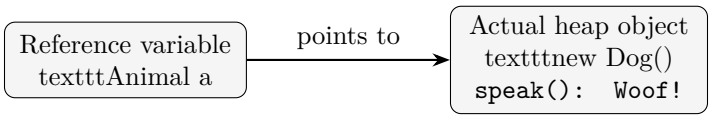
Polymorphism means "many forms." In Java, a superclass reference can refer to an instance of a subclass. The *reference type* determines which members you may call; the *actual object type* determines which overridden instance-method body runs.

```
Animal a = new Dog();
a.speak(); // Woof!

a = new Cat();
a.speak(); // Meow!
```

The variable `a` has declared type `Animal`, but the first object is a `Dog` and the second is a `Cat`. Java chooses the overridden `speak()` implementation at runtime. This is called **dynamic dispatch**.

5.1 A Reference Type Is Not the Object Type




```
class Dog extends Animal {
    @Override
    public void speak() { System.out.println("Woof!"); }
    public void fetch() { System.out.println("Fetching"); }
}
Animal a = new Dog();
a.speak(); // Allowed: Animal declares speak()
// a.fetch(); // Compile error: Animal does not declare fetch()
```

Even though the actual object is a dog, `a.fetch()` is not allowed because the declared reference type is `Animal`. The compiler checks accessible methods using the reference type.

5.2 Upcasting, Downcasting, and `instanceof`

Assigning a child reference to a parent variable is **upcasting** and happens automatically. Casting a parent reference back to a child type is **downcasting** and requires an explicit cast.

```
Animal a = new Dog(); // Upcast: automatic
if (a instanceof Dog) {
    Dog d = (Dog) a; // Downcast: explicit
    d.fetch();
}
```

A cast does not transform an object into another kind of object. It only changes how a reference is viewed. Attempting to cast an actual `Cat` object to `Dog` produces `ClassCastException`. Check with `instanceof` when necessary; better class designs often avoid frequent casts.

6 Why Polymorphism Is Useful: Arrays of Different Subclasses

Suppose we want to make every animal speak. Without polymorphism, we might keep separate arrays of dogs and cats. With a common parent, one array can hold both:

```
Animal[] animals = {
    new Dog(),
    new Cat(),
    new Dog()
};

for (Animal animal : animals) {
    animal.speak();
}
// Woof!
// Meow!
// Woof!
```

The loop never asks "Is this a dog? Is this a cat?" It simply calls the method promised by `Animal`. Adding another animal type does not require changing this loop, as long as that type provides the appropriate behavior.

7 Interfaces: A Contract for Behavior

Inheritance organizes related classes in an *is-a* hierarchy. But unrelated classes can share capabilities. A phone, an electric car, and a laptop can all be charged. It would be strange to force them all to extend one `Vehicle` superclass. Instead, Java lets us describe the capability using an **interface**.

```
interface Chargeable {
    void charge(); // public abstract is implicit here
}

class Phone implements Chargeable {
    @Override
    public void charge() {
        System.out.println("Charging phone");
    }
}

class ElectricCar implements Chargeable {
    @Override
    public void charge() {
        System.out.println("Charging car");
    }
}
```

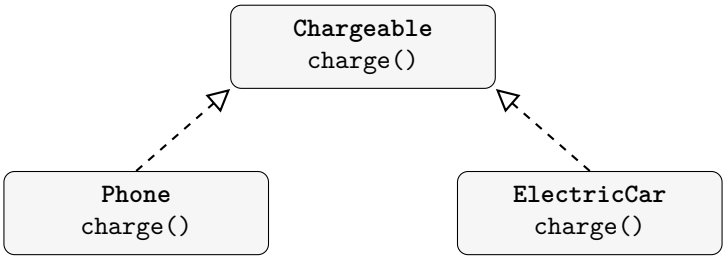
An interface is a **contract**: a class that implements it promises to provide the required methods, unless the implementing class is abstract. In this beginner example, the interface declares `charge()` without an implementation; the implementing classes supply the method bodies.

```
Chargeable device = new Phone();
device.charge(); // Charging phone

device = new ElectricCar();
device.charge(); // Charging car
```

Notice that the variable is declared with the interface type, `Chargeable`. This is polymorphism again, but now through an interface rather than a parent class.

7.1 Visualizing an Interface Contract



Dashed arrows represent implementing an interface. Neither `Phone` nor `ElectricCar` needs to inherit from the other.

7.2 A More Practical Interface: `Payable`

```
interface Payable {
    double calculatePay();
}

class HourlyWorker implements Payable {
    private double hours;
    private double rate;

    public HourlyWorker(double hours, double rate) {
        this.hours = hours;
        this.rate = rate;
    }

    @Override
    public double calculatePay() {
        return hours * rate;
    }
}

class FixedContract implements Payable {
    private double total;

    public FixedContract(double total) {
        this.total = total;
    }

    @Override
    public double calculatePay() {
        return total;
    }
}
```

```
public static double totalPayroll(Payable[] workers) {
    double total = 0;
    for (Payable worker : workers) {
        total += worker.calculatePay();
```

```
    }
    return total;
}
```

The method works for any objects implementing `Payable`, even if their internal calculations differ. This is a major reason programmers use interfaces: callers depend on the *behavior promised*, not the specific class.

8 Interface Rules Students Should Know

- A class uses `implements` for interfaces and `extends` for a superclass.
- A class can extend only *one* class, but can implement *multiple* interfaces.
- You cannot directly instantiate an interface: `new Chargeable()` is invalid.
- Interface methods declared without a body are implicitly `public abstract`; implementing methods must be `public`.
- Interface fields are implicitly `public static final` constants, not per-object instance fields.
- Modern Java interfaces may also contain `default` and `static` methods with bodies. For CMSC131, first master abstract method declarations and `implements`.

```
interface Printable {
    void print();
}
interface Saveable {
    void save();
}
class Report implements Printable, Saveable {
    @Override
    public void print() { System.out.println("Printing"); }

    @Override
    public void save() { System.out.println("Saving"); }
}
```

`Report` can satisfy two contracts at once. It still does not extend two classes; Java does not support multiple class inheritance.

9 Abstract Classes: Between Regular Classes and Interfaces

Sometimes we know every animal should have a `speak()` method, but we do not have a sensible default sound. An **abstract class** can provide shared fields and completed methods while requiring subclasses to implement certain methods.

```
abstract class Animal {
    private String name;

    public Animal(String name) { this.name = name; }
    public String getName() { return name; }
```

```
    public abstract void speak(); // No method body
}

class Dog extends Animal {
    public Dog(String name) { super(name); }

    @Override
    public void speak() { System.out.println("Woof!"); }
}
```

`Animal` cannot be directly instantiated because it is abstract: `new Animal("Milo")` is invalid. But `Animal a = new Dog("Milo");` is valid. Abstract classes can have constructors, instance fields, concrete methods, and abstract methods. A non-abstract subclass must implement any remaining abstract methods.

9.1 Class Versus Abstract Class Versus Interface

	Regular class	**Abstract class**	**Interface**
Instantiate directly?	Yes, if accessible constructor	No	No
Instance fields?	Yes	Yes	No
Constructors?	Yes	Yes	No
Method bodies?	Yes	Yes	Yes, e.g. default methods
Used with	`new`, `extends`	`extends`	`implements`

Use a superclass when classes genuinely share an *is-a* relationship and common state or behavior.
Use an interface when you want to promise a capability across potentially unrelated classes.

10 The `Object` Class: A Parent of Every Class

Every Java class ultimately inherits from `java.lang.Object`, directly or indirectly. That is why objects have methods such as `toString()` and `equals(Object)`, even when you did not write them yourself.

```
class Book {
    private String title;
    public Book(String title) { this.title = title; }

    @Override
    public String toString() {
        return "Book: " + title;
    }
}

Book b = new Book("Java Notes");
System.out.println(b); // Book: Java Notes
```

`println` calls the object's `toString()` to get a readable representation. Without an override, `Object.toString()` generally produces a class name and hash-like identifier, not the fields students expect.

10.1 Reference Equality Versus Logical Equality

For object references, `==` asks whether two references refer to the *same object*. The `equals()` method can be overridden to compare meaningful content.

```
String a = new String("hi");
String b = new String("hi");
System.out.println(a == b); // false: different objects
System.out.println(a.equals(b)); // true: same characters
```

When overriding `equals`, the correct signature is `public boolean equals(Object obj)`. A method named `equals(Book other)` *overloads* rather than overrides `Object.equals(Object)`.

```
import java.util.Objects;

class Book {
    private String title;
    public Book(String title) { this.title = title; }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Book)) return false;
        Book other = (Book) obj;
        return Objects.equals(title, other.title);
    }

    @Override
    public int hashCode() {
        return Objects.hash(title);
    }
}
```

The `hashCode()` override matters because Java requires equal objects to have equal hash codes. The details of hashing are beyond this chapter, but **override `hashCode()` whenever you override `equals()`**. Note that equality can be designed differently depending on the class's meaning; this example treats two books with the same title as equal.

11 Static Methods and Fields Are Not Dynamically Overridden

Dynamic dispatch applies to overridden *instance methods*. Static methods belong to classes, not objects, so a same-signature static method in a subclass *hides* the superclass method rather than overriding it.

```
class Parent {
    public static void identify() { System.out.println("Parent"); }
    public void speak() { System.out.println("Parent speaks"); }
}
class Child extends Parent {
    public static void identify() { System.out.println("Child"); }
    @Override
    public void speak() { System.out.println("Child speaks"); }
}

Parent p = new Child();
p.speak(); // Child speaks: actual object type
```

```
p.identify(); // Parent: reference type (avoid this style)
```

Call static methods through the class name (`Parent.identify()`) to make the intended behavior clear. Fields are also not polymorphically overridden: if a child declares a field with the same name, it *hides* the parent field.

12 A Complete Example: Shapes

We can now combine abstract classes, constructors, inheritance, overriding, and polymorphism. The abstract parent provides a shared color, but each shape calculates its area differently.

```
abstract class Shape {
    private String color;

    public Shape(String color) { this.color = color; }
    public String getColor() { return color; }
    public abstract double area();
}

class Rectangle extends Shape {
    private double width, height;

    public Rectangle(String color, double w, double h) {
        super(color);
        width = w;
        height = h;
    }

    @Override
    public double area() { return width * height; }
}

class Circle extends Shape {
    private double radius;

    public Circle(String color, double radius) {
        super(color);
        this.radius = radius;
    }

    @Override
    public double area() {
        return Math.PI * radius * radius;
    }
}
```

```
Shape[] shapes = {
    new Rectangle("Blue", 3, 4),
    new Circle("Red", 2)
};

for (Shape s : shapes) {
    System.out.printf("%s area: %.2f%n",
                      s.getColor(), s.area());
}
// Blue area: 12.00
```

```
// Red area: 12.57
```

Trace the first iteration: The variable `s` has declared type `Shape`, so calling `getColor()` and `area()` is legal. The actual object is a `Rectangle`, so the rectangle's `area()` executes. On the second iteration, the actual object is a `Circle`, so the circle's `area()` executes. The loop itself stays unchanged.

13 Common Mistakes and How to Debug Them

13.1 Mistake 1: Calling a Child-Only Method Through a Parent Reference

```
Animal pet = new Dog();
// pet.fetch(); // Does not compile if Animal has no fetch()
```

Fix by calling methods declared in the parent, redesigning the shared interface, or using a checked downcast when genuinely necessary.

13.2 Mistake 2: Forgetting the Required Parent Constructor

```
class Parent {
    public Parent(int x) { }
}
class Child extends Parent {
    // public Child() { } // Error: implicit super() invalid
    public Child() { super(10); } // Correct
}
```

13.3 Mistake 3: Accidentally Overloading Instead of Overriding

```
class Animal {
    public void speak() { }
}
class Dog extends Animal {
    // Not an override; parameter list differs!
    public void speak(int times) { }
}
```

If the intention was overriding, match the method signature exactly and add `@Override`.

13.4 Mistake 4: Making an Interface Implementation Nonpublic

```
interface Playable { void play(); }
class Game implements Playable {
    // void play() { } // Error: weaker access than public
    public void play() { System.out.println("Play!"); }
}
```

13.5 Mistake 5: Assuming a Cast Changes the Object

```
Animal pet = new Cat();
// Dog dog = (Dog) pet; // ClassCastException at runtime
```

The actual object is still a `Cat`. A cast cannot turn it into a `Dog`.

14 Tracing Practice with Answers

14.1 Trace A: Which Method Runs?

```
class A {
    public void one() { System.out.print("A1 "); }
    public void two() { System.out.print("A2 "); }
}
class B extends A {
    @Override
    public void one() { System.out.print("B1 "); }
    public void three() { System.out.print("B3 "); }
}
A obj = new B();
obj.one();
obj.two();
// obj.three(); // Does this compile?
```

Answer: `B1 A2`. The actual object is `B`, so its override of `one()` runs. Since `B` does not override `two()`, the inherited `A.two()` runs. `obj.three()` does not compile because the declared reference type `A` has no `three()` method.

14.2 Trace B: Constructor and Override

```
class Parent {
    public Parent() { System.out.print("P "); }
    public void show() { System.out.print("Parent "); }
}
class Child extends Parent {
    public Child() { System.out.print("C "); }
    @Override
    public void show() { System.out.print("Child "); }
}
Parent x = new Child();
x.show();
```

Answer: `P C Child`. The superclass constructor runs first, then the subclass constructor. After construction, the overridden method on the actual `Child` object executes.

14.3 Trace C: Interface References

```
interface Greeter { void greet(); }
class English implements Greeter {
    public void greet() { System.out.print("Hello "); }
```

```
}
class Spanish implements Greeter {
    public void greet() { System.out.print("Hola "); }
}
Greeter[] g = {new English(), new Spanish()};
for (Greeter item : g) item.greet();
```

Answer: `Hello Hola`. The array holds references to different objects, each implementing the same interface contract.

15 Final Reference: Key Rules

Term	What to remember
`extends`	Creates an is-a relationship between classes.
`implements`	Promises to provide an interface's required methods.
`super(...)`	Calls a parent constructor.
`super.method()`	Calls the parent's method implementation.
`@Override`	Asks the compiler to verify an override.
Overloading	Same name, different parameter list.
Overriding	Subclass replaces an inherited instance-method implementation.
Polymorphism	A common reference type can point to different object types.
Dynamic dispatch	Runtime object type determines overridden instance-method behavior.
Abstract class	Cannot instantiate directly; can share state and require methods.
Interface	Describes behavior that implementing classes promise.
`instanceof`	Checks an object's runtime type before a cast.
`Object`	Root superclass providing methods such as `toString()` and `equals()`.

The single most useful mental model: The declared reference type determines *which methods Java lets you call*; the actual object type determines *which overridden instance-method implementation runs.*